Five-Fabric Capability Audit

Question asked: *"Prove that TECP + EQF + Intelligent Fabric + Case Resolution

- Forward/Reverse are one executable system in the current repository, without

creating any duplicate engine."*

Executable form: `pkg/assurance/fabric.go` · asserted by `pkg/assurance/fabric_test.go` **Vocabulary:** `docs/architecture/VERIQO_CANONICAL_VOCABULARY.md`

---

1. What was audited

Each of the five fabrics, along the eleven dimensions the question demands:

```
CAPABILITY → CANONICAL PACKAGE → ENTRY POINT → CALL GRAPH → DATA FLOW
  → STATE FLOW → EVIDENCE FLOW → TEST → E2E TEST → REPLAY → FAIL-CLOSED
```

Every dimension is a sentence naming something real, never a tick. A tick would tell a reader nothing they could check. `FabricAudit.Incomplete()` returns the blanks, and `TestAllFiveFabricsAreAuditedOnEveryDimension` fails the build if any fabric ships one.

Two further checks make the audit adversarial rather than self-congratulatory:

- `TestEveryCanonicalPackageExists` resolves every package path named in every row against the filesystem. An audit pointing at a package that is not there would be worse than a blank.
- `TestEveryFabricNamesItsDuplicationRisk` requires each fabric to name the packages that **look like** a rival authority, and say why each is not one. Those are the packages a reviewer will find, so the audit names them first.

---

2. The answer

Yes, with one qualification that matters.

The five fabrics are one executable system: each has a real entry point, a real call graph through named packages, a real state flow, a durable evidence trail in the one audit ledger, unit tests, an end-to-end proof, a replay path, and a fail-closed behaviour stated as a refusal rather than a log line.

The qualification is this: **being one executable system is an engineering property, not an assurance one.** The audit proves the fabrics compose. It does not prove they compose *correctly* against real matters, real sources or a real tribunal — see `pkg/assurance`'s second axis, where every one of these capabilities sits at INTERNALLY_PROVED and none at EXTERNALLY_VALIDATED.

---

3. Duplication risks, named

The honest part of the audit. For each fabric, what looks like a second authority on the same question, and why it is not:

Fabric	Looks like a rival	Why it is not
TECP	`pkg/insurance/auditlink`, `insurance/decision.AppendToLedger`, `insurance/action.AppendToLedger`	All three write into the one `audit.AuditStore`. There is a single `Append` primitive in the repository; everything else mirrors into it
EQF	`pkg/governance/qualification`	It records *operational* qualification decisions. It does not compute epistemic state, so the two are not a second opinion on one question
IF	`pkg/ucr`, `pkg/kernel/reasoning`	Both reason. Neither can produce a finding: only `pkg/proof` can, and only from a sealed, sufficient object
CRF	`pkg/insurance/case`, `casestate`, `caseroom`, `cre`	Projections, not rivals. `casestate`'s fourteen states are mapped onto the nine canonical phases, and `TestEveryInsuranceStateMapsOntoTheFabric` fails if a state is added without a mapping
FREF	`pkg/workflow`, `pkg/kernel/execgraph`	Both orchestrate. `fref` does not: it is a contract that refuses an execution that skipped or reordered a stage, and names the package each stage must run in

---

4. The audit, in full

```
=== TECP ===
  CAPABILITY             Hold the canonical state of evidence, who may touch it, and what was done to it
  CANONICAL PACKAGE      veriqo/pkg/evidence/manifest, veriqo/pkg/evidence/provenance, veriqo/pkg/authz, veriqo/pkg/security/identity, ver
  ENTRY POINT            manifest.Registry (evidence), audit.AuditStore.Append (record), authz (permission)
  CALL GRAPH             ingest -> manifest.Registry.Submit -> authz policy check -> custody event -> audit.AuditStore.Append -> WAL
  DATA FLOW              raw bytes -> content hash -> evidence version -> custody chain head -> canonical audit record
  STATE FLOW             DRAFT -> SUBMITTED -> FINALIZED -> (SUPERSEDED). FINALIZED is structurally terminal for mutation
  EVIDENCE FLOW          hash-linked audit records with a Merkle root, plus the custody chain on each version
  TEST                   pkg/evidence/manifest, pkg/platform/audit, pkg/authz, pkg/platform/timestamp
  E2E TEST               test/e2e/eight_blockers, test/integration
  REPLAY                 pkg/replay ManifestAdapter restores state from the record; audit.Auditor.VerifyChain re-derives the ledger
  FAIL-CLOSED BEHAVIOR   evidence that cannot be pinned to a version and a content hash never enters; a rights failure denies before conta
  DUPLICATION RISK       pkg/insurance/auditlink looks like a second ledger and is not: it mirrors into the one AuditStore. pkg/insurance/
  RETIRES                Evidence Engine, Evidence Fabric, Trust Engine, Trust Kernel, Unified Evidence

=== EQF ===
  CAPABILITY             Decide what the evidence supports, what it does not, and what is missing
  CANONICAL PACKAGE      veriqo/pkg/qualification/state, veriqo/pkg/qualification/independence, veriqo/pkg/qualification/observability, ve
  ENTRY POINT            reverseproof.Build (obligations), independence.Assess (sources), state.New (verdict)
```

```
  CALL GRAPH             claim -> reverseproof.Build -> reverseproof.Analyze -> independence.Cluster -> state.New -> nextbest.Rank
  DATA FLOW              claim + conditions -> requirements -> gap -> effective source count -> qualification state -> ranked candidates
  STATE FLOW             the ten qualification states; there is no PROVEN state and Parse refuses one by name
  EVIDENCE FLOW          the qualification record carries its policy version, rationale and material dissent
  TEST                   pkg/qualification/* (72 tests)
  E2E TEST               test/adversarial constitutional suite, pkg/proof sufficiency derivation
  REPLAY                 qualification is a pure function of its inputs; re-running Analyze on the same set reproduces the gap
  FAIL-CLOSED BEHAVIOR   UNKNOWN independence is never INDEPENDENT; an unattempted requirement is never observed-absent; a rights-denied c
  DUPLICATION RISK       pkg/governance/qualification predates this fabric and records operational qualification decisions; it does not co
  RETIRES                EQF, Qualification, epistemic layer

=== IF ===
  CAPABILITY             Propose explanations. Intelligence proposes; it never concludes
  CANONICAL PACKAGE      veriqo/pkg/moat/kg, veriqo/pkg/moat/causal, veriqo/pkg/moat/hbayes, veriqo/pkg/moat/temporal, veriqo/pkg/moat/eco
  ENTRY POINT            moat/intelligence, moat/causal hypothesis construction
  CALL GRAPH             evidence graph -> entity resolution -> causal structure -> hypothesis set -> (stops)
  DATA FLOW              evidence + entities -> knowledge graph -> hypotheses with cited inputs -> inference traces
  STATE FLOW             hypothesis status only; a hypothesis never becomes a finding inside this fabric
  EVIDENCE FLOW          pkg/inference.InferenceTrace pins every input a hypothesis rests on
  TEST                   pkg/moat/* package tests
  E2E TEST               the knowledge/intelligence boundary test: intelligence yields a hypothesis, never a finding
  REPLAY                 inference traces cite their inputs, so a hypothesis is re-derivable from the same evidence
  FAIL-CLOSED BEHAVIOR   reasoning that cannot cite its inputs produces no hypothesis; an entity resolution below threshold stays unresolv
  DUPLICATION RISK       pkg/ucr (Unified Cognitive Reasoning) and pkg/kernel/reasoning both reason. They are not a second authority becau
  RETIRES                Intelligence Layer, Intelligent Fabric, Knowledge Fabric, moat

=== CRF ===
  CAPABILITY             Hold one case across every domain, from identity to outcome
  CANONICAL PACKAGE      veriqo/pkg/casefabric, veriqo/pkg/proof, veriqo/pkg/lineage, veriqo/pkg/insurance/casestate
  ENTRY POINT            casefabric.Open, then SetScope, AddEvidence, AddHypothesis, RegisterClaim, AttachProof, Resolve
  CALL GRAPH             Open -> SetScope -> AddEvidence -> AddHypothesis -> RegisterClaim -> BeginQualification -> AttachProof (verifies
  DATA FLOW              identity + scope -> pinned evidence -> hypotheses -> claims -> sealed proof objects -> outcome
  STATE FLOW             nine canonical phases; every domain state maps onto one, and an unmapped state is refused
  EVIDENCE FLOW          a hash-chained case timeline, mirrored into the one audit store by casefabric.Mirror
  TEST                   pkg/casefabric (32 tests), pkg/proof (51 tests)
  E2E TEST               test/integration internal-gaps suite, test/adversarial fabric suite
  REPLAY                 casefabric.VerifyTimeline re-derives the chain; proof.VerifyHash re-derives every attached object
  FAIL-CLOSED BEHAVIOR   a case cannot resolve past an unproven material claim or an untested rival hypothesis; an outcome that adjudicate
  DUPLICATION RISK       pkg/insurance/case, casestate, caseroom and cre are insurance's own case machinery and look like a rival engine.
  RETIRES                Case Engine, Case Resolution Engine, Case lineage, case workflow

=== FREF ===
  CAPABILITY             Run both directions over the same evidence, and prove they close
  CANONICAL PACKAGE      veriqo/pkg/fref, veriqo/pkg/workflow, veriqo/pkg/execution, veriqo/pkg/kernel/runtime
  ENTRY POINT            fref.NewExecution(Forward|Reverse, subject), then Complete per stage; fref.Close for the closure
  CALL GRAPH             forward: OBSERVATION -> EVIDENCE -> KNOWLEDGE -> REASONING -> TRUST -> FINDING -> DECISION. reverse: CLAIM -> PRO
  DATA FLOW              each stage pins an output hash; the closure compares the forward evidence set against the reverse required set
  STATE FLOW             stages complete in canonical order and once only; a run is complete only at its terminal stage
  EVIDENCE FLOW          stage records carry the package that ran, the tick and the pinned output
  TEST                   pkg/fref (26 tests)
  E2E TEST               test/adversarial fabric closure cases
  REPLAY                 an execution is a record of pinned outputs, so a replay re-runs each stage against the same inputs
  FAIL-CLOSED BEHAVIOR   a stage cannot complete before its predecessors; a reverse run that stops at QUALIFICATION is incomplete; a closu
  DUPLICATION RISK       pkg/workflow and pkg/kernel/execgraph both orchestrate. fref does not orchestrate: it is a contract that refuses
  RETIRES                Execution, Orchestrator, pipeline, universal workflow
```

---

5. What the audit does not establish

- That any fabric behaves correctly under real load, real adversaries or real data. Every E2E test in the table is VERIQO's own.
- That the duplication risks named in §3 are the only ones. They are the ones this audit found; a reviewer may find others, and the rows are written to be contested rather than believed.
- That the fail-closed behaviours hold under every failure mode. They hold under the ones the tests exercise.